

PdfPecker manual

Fast HTML and Markdown to PDF for .NET, with no dependencies.

For PdfPecker 1.0.0 · pdfpecker.net/docs

Contents

PdfPecker 1.0.0 . . . . . **3**
Install 3
How it compares 3
System requirements 5
Usage 6
Merging PDFs 10
PDF to image 10
Website 11
What is supported 12
Limitations 15
Performance 17
Building 18
How it works 18

PdfPecker

Fast HTML and Markdown to PDF conversion, and PDF to PNG rendering, for .NET, with no dependencies.

PdfPecker has its own HTML parser, Markdown parser, CSS engine, layout engine and PDF writer, and its own PDF renderer. It does not start a browser, ship native binaries or pull in third-party packages; the NuGet package depends only on the .NET base class library.

```
using PdfPecker;

byte[] pdf = HtmlToPdf.Convert("<h1>Hello</h1><p>World</p>");
File.WriteAllBytes("hello.pdf", pdf);
```

Install

```
dotnet add package PdfPecker
```

Until the package is published to nuget.org, build it locally and add the folder as a package source:

```
dotnet pack src/PdfPecker -c Release -o ./nupkg
dotnet nuget add source "$(pwd)/nupkg" --name pdfpecker-local
dotnet add <your project> package PdfPecker
```

The package targets `net10.0` and works on .NET 10 and later.

How it compares

The same three documents converted to A4 PDF by PdfPecker and four free .NET alternatives, on one 4-core cloud VM (Intel Xeon @ 2.8 GHz, 16 GB, Linux, .NET 10.0.12). Times are medians after a 3-second warm-up; "first PDF" includes starting the browser where there is one. Parallel runs one conversion per core at once on the 5-page document. Peak memory is the proportional set size (PSS) of the whole process tree, so a browser's helper processes are included once and shared pages are not double-counted.

Converter	Engine	To deploy	First PDF	Invoice, 1 page	Feature tour, 5 pages	Report, 30 pages	Parallel	Peak memory
PdfPecker	own, managed	3.1 MB DLL	0.27 s	**0.86 ms**	**2.7 ms**	**39 ms**	**962 docs/s**	**83 MB**
HtmlRenderer.PdfSharp 1.6.1	own, managed (CSS 2)	2.4 MB of DLLs	0.47 s	5.0 ms	16 ms	473 ms	88 docs/s	255 MB

Converter	Engine	To deploy	First PDF	Invoice, 1 page	Feature tour, 5 pages	Report, 30 pages	Parallel	Peak memory
wkhtmltopdf 0.12.6	Qt WebKit, native	about 90 MB with Qt	0.31 s	289 ms	333 ms	693 ms	12 docs/s	164 MB
PuppeteerSharp 25.13	Chromium 141	about 600 MB browser	0.91 s	128 ms	166 ms	539 ms	12 docs/s	849 MB
Microsoft.Playwright 1.63	Chromium 141	browser + 135 MB driver	1.08 s	142 ms	191 ms	522 ms	11 docs/s	986 MB

How to read it:

- PdfPecker is 60 to 150 times faster than the Chromium-based converters on the 1 and 5-page documents and about 14 times faster on the 30-page report, and uses a tenth of their memory. Per-document overhead (a browser tab, a process start) dominates for the others on small documents.
- Output is not identical. Chromium is the reference for modern CSS and runs JavaScript, which PdfPecker does not. HtmlRenderer supports only HTML 4 and CSS 2 (it drops the invoice's table header colours), and wkhtmltopdf uses a 2016 WebKit, shrinks content to fit by default (24 pages for the report instead of 29) and was archived in 2023.
- PDF sizes for the 1 / 5 / 30-page documents: PdfPecker 12 / 26 / 126 KB, Chromium 72 / 139 / 171 KB, wkhtmltopdf 30 / 59 / 191 KB, HtmlRenderer 35 / 65 / 273 KB.
- Each converter was used the way its documentation suggests for many documents: PdfPecker and HtmlRenderer in process, wkhtmltopdf as one process per document, and PuppeteerSharp and Playwright with one shared browser and a new tab per document (Playwright in one shared browser context, tagged PDF output off for both).

Commercial converters were not measured here. One user compared PdfPecker with **IronPDF 2026.7.2** (Chrome-based) on the 26 templates of their production app, on a 22-core development machine after warm-up: IronPDF took 13.6 s for all 26 documents one at a time and PdfPecker 17 ms (about 800 times faster), and with 22 in parallel IronPDF made 9.2 documents/s and PdfPecker 1,143 (about 125 times). 24 of the 26 documents came out identical; the other two differ by less than 0.25 mm.

To reproduce the table (needs wkhtmltopdf on `PATH` and a Chromium at `CHROMIUM_PATH`):

```
dotnet run --project bench/PdfPecker.Comparison -c Release
```

The comparison project is not part of `PdfPecker.sln`, so its third-party packages never reach the library or its tests. Your numbers will differ with hardware and documents.

System requirements

Required: the .NET 10 runtime (or later). Nothing else: the library is pure managed code with no native binaries, browser or third-party packages, so it runs wherever .NET 10 runs (Windows, Linux, macOS; x64 and ARM64).

Recommended:

- Register the fonts your templates use with `options.Fonts` (see Fonts); other fonts fall back to Helvetica, Times or Courier metrics.
- Memory and CPU are small. Measured on a 4-core 2.1 GHz Xeon cloud VM (Linux, .NET 10, Release), each document in a fresh process:

Document	Pages	Time, warm	Time, first in process	Peak process memory	Smallest heap that works
Payslip, invoice, voucher	1 to 3	1 to 3 ms	0.2 s	50 MB	under 16 MB
Same with PDF/A or embedded fonts	1 to 3	3 to 5 ms	0.2 s	56 MB	under 16 MB
Report with tables	30	30 to 40 ms	0.3 s	75 MB	under 16 MB
Report with tables	1,000	1.7 s	3 s	480 MB	245 MB

  The process itself starts at about 32 MB; a conversion uses about one core, so throughput grows with cores. PDF/A and font embedding add a few milliseconds and about 5 MB for the embedded font data. Memory grows roughly with page count, so plan about 0.5 MB per page for very long documents.
- For the website and API container, give it about 2 GB (`--memory 2g`). It runs one conversion per 512 MB of its memory limit (up to one per core) and refuses documents over 1,000 pages, so 2 GB lets four large documents convert at once. Behind a proxy, set `TrustedProxies` (see Website).

`dotnet run --project bench/PdfPecker.Benchmarks -c Release -- --requirements [your.html ...]` measures this on your own machine and documents.

Usage

```
// String to bytes
byte[] pdf = HtmlToPdf.Convert(html);

// String to a stream
using (var file = File.Create("out.pdf")) HtmlToPdf.Convert(html, file);

// ASP.NET Core
app.MapGet("/invoice", () => Results.File(HtmlToPdf.Convert(html), "application/pdf",
"invoice.pdf"));

// File to file; relative  and <link> paths resolve against the HTML file's folder
HtmlToPdf.ConvertFile("invoice.html", "invoice.pdf");

// Off the calling thread
byte[] pdf = await HtmlToPdf.ConvertAsync(html);
```

All methods are static and thread-safe, so you can convert many documents in parallel.

Markdown

```
// Markdown to PDF, styled like a GitHub README
byte[] pdf = MarkdownToPdf.Convert(File.ReadAllText("README.md"));

// File to file; relative image paths resolve against the Markdown file's folder
MarkdownToPdf.ConvertFile("README.md", "README.pdf", new PdfOptions { FooterText = "Page
{page} of {pages}" });

// Markdown to HTML: a body fragment, or a complete page with the stylesheet
string fragment = MarkdownToHtml.Convert("# Hello\n\n*World*");     // <h1
id="hello">Hello</h1>\n<p><em>World</em></p>
string page = MarkdownToHtml.ConvertDocument(markdown);
MarkdownToHtml.ConvertFile("README.md", "README.html");
```

`MarkdownOptions` switches the GitHub extensions off for strict CommonMark (`GitHubExtensions = false`), drops heading ids (`HeadingIds = false`), shows raw HTML as text (`AllowRawHtml = false`), replaces the stylesheet (`StyleSheet`, see `MarkdownToHtml.DefaultStyleSheet`) or sets the title (default: the first heading). `MarkdownToPdf` takes `PdfOptions` and `MarkdownOptions` side by side.

Options

```
var options = new PdfOptions
{
    PageSize = PageSize.Letter.Landscape,            // A3, A4 (default), A5, Letter, Legal or
new PageSize(w, h) in points
    Margins = PageMargins.FromMillimeters(20, 15, 20, 15),
    Scale = 1,                                       // like Chrome's print scale; 0.99916
matches IronPDF's A4 output
    BaseDirectory = "/var/app/templates",            // where relative image/stylesheet paths
are resolved
    ResourceResolver = url => myCache.Get(url),      // serve http(s) or embedded resources
yourself; null = default handling
    UserStyleSheet = "body { font-size: 10pt }",     // extra CSS applied after the
document's own
    FooterText = "Page {page} of {pages}",           // also HeaderText
    Title = "Invoice 42", Author = "Accounts",       // document properties (Title defaults
to <title>)
    Keywords = "invoice", Creator = "MyApp",         // also Subject, and Producer (default
"PdfPecker")
    CustomProperties = { ["RunId"] = "42" },         // your own properties, in the PDF's
document information
    Bookmarks = true,                                // PDF bookmarks from h1-h6 (default)
    Compress = true,                                 // Flate-compress page content (default)
    AllowFileAccess = false,                         // true: also read files outside
BaseDirectory (trusted HTML only)
    MaxPages = 1000,                                 // refuse longer documents with
PdfLimitException (default 100,000)
};
byte[] pdf = HtmlToPdf.Convert(html, options);
```

Local images and stylesheets are read only from inside `BaseDirectory` (the input file's folder for `ConvertFile`): absolute paths, `file:` URLs, `..` and symbolic links that lead outside it are ignored unless `AllowFileAccess = true`, and converting a string without a `BaseDirectory` reads no files at all. For HTML from untrusted users, keep it that way, set a low `MaxPages`, and pass a cancellation token with a timeout (`HtmlToPdf.Convert(html, options, token)`): cancelling stops the conversion where it is.

A CSS `@page { size: A4 landscape; margin: 15mm }` rule overrides the page size and margins unless `IgnoreCssPageRules` is set. PdfPecker never makes network requests on its own: `http(s)` images and stylesheets are only loaded through `ResourceResolver`.

Fonts

Register TrueType or OpenType fonts and use them by name in CSS. Only the glyphs a document uses are embedded, and text can use any Unicode character the fonts have (Greek, Cyrillic, Central European, CJK, ...):

```
var options = new PdfOptions();
options.Fonts.AddFile("fonts/OpenSans-Regular.ttf")
             .AddFile("fonts/OpenSans-Bold.ttf");             // family, weight and style are
read from the files
options.Fonts.Add(bytes, familyName: "sans-serif");           // or override a generic family
options.Fonts.AddDirectory(@"C:\Windows\Fonts");               // read once, loaded only when
used
byte[] pdf = HtmlToPdf.Convert("<p style=\"font-family: 'Open Sans'\">Łódź, Ελλάδα,
Москва</p>", options);
```

Documents can bring their own fonts with `@font-face` (files, `data:` URIs or `ResourceResolver` URLs; `local()` refers to registered fonts):

```
@font-face { font-family: Brand; src: url(fonts/brand.woff2) format("woff2"); font-weight:
400; }
@font-face { font-family: Brand; src: url(fonts/brand-bold.ttf); font-weight: 700; unicode-
range: U+0000-024F; }
```

A `FontCollection` can be shared between options and parallel conversions. Formats: `.ttf`, `.otf`, `.ttc` collections, WOFF and WOFF2 (including WOFF2 collections; decoded with .NET's built-in Brotli). Fonts are matched like browsers do: by family, then style, then the nearest weight; a missing bold or italic face is synthesized. A character that the chosen font lacks is drawn with the next font in the `font-family` list that has it, then the standard font, then any other registered font.

Text without a registered font uses the standard PDF fonts (Helvetica, Times, Courier), which viewers supply themselves, so files stay small but are limited to Western European characters. Set `EmbedStandardFonts = true` to embed the bundled Liberation fonts in their place instead: they have the same widths, so the layout does not change, the text looks the same in every viewer, and any character they cover (Latin, Greek, Cyrillic, ...) works. PDF/A always does this. It adds a subset of each font used to the file (typically 10-25 KB per font).

A family that no font provides falls back silently, like in a browser. To notice, set `MissingFont`: it is called once per conversion for each family that had to be substituted, and throwing from it fails the conversion. To use the fonts installed on the machine for families you have not registered, set `UseSystemFonts = true` (off by default, so the output does not depend on the machine):

```
options.MissingFont = family => logger.LogWarning("Font {Family} is not available", family);
options.UseSystemFonts = true;   // looks in C:\Windows\Fonts, /Library/Fonts,
/usr/share/fonts, ...
```

Watermarks, passwords and PDF/A

```
var options = new PdfOptions
{
    // A diagonal stamp on every page. Also: Image (PNG/JPEG bytes), FontSize, LetterSpacing,
Angle, Opacity, Bold, Behind.
    Watermark = new PdfWatermark { Text = "WATERMARK", Color = "#c00000", Opacity = 0.25f },

    // AES-256 encryption (PDF 2.0 / Acrobat X and later). Without an OwnerPassword a random
one is used,
    // so the permissions cannot be lifted.
    Security = new PdfSecurity
    {
        UserPassword = "1234",                                  // needed to open the file;
null = opens freely
        OwnerPassword = "owner-secret",                         // full access
        Permissions = PdfPermissions.Print | PdfPermissions.Copy,
    },
};
```

For archiving, set `Conformance = PdfConformance.PdfA2b` (or `PdfA3b`): the PDF gets XMP metadata, an sRGB output intent and a file ID, and CMYK JPEGs are converted to RGB. All fonts are embedded: text without a registered font uses the bundled Liberation fonts (see Fonts). PDF/A does not allow encryption, so combining `Conformance` with `Security` throws an `ArgumentException`. AES-256 keys come from a password hash that is slow on purpose, which adds about 3 ms to a conversion.

Digital signatures

Sign the PDF with a certificate (a `.pfx` file or one from the certificate store). Adobe Reader then shows who signed it and whether the file was changed afterwards.

```
var cert = X509CertificateLoader.LoadPkcs12FromFile("signer.pfx", "pfx-password");
var options = new PdfOptions
{
    Signature = new PdfSignatureOptions(cert)
    {
        Reason = "Approved",
        Location = "Copenhagen",
        // Optional: a visible box (points from the top-left corner; Page = -1 is the last
page).
        Appearance = new PdfSignatureAppearance { Page = -1, X = 340, Y = 720, Width = 220,
Height = 70 },
        // Optional: a trusted RFC 3161 timestamp (PAdES B-T). The only option that uses the
network.
        TimestampUrl = "http://timestamp.digicert.com",
    },
};

// Or sign any existing PDF; earlier signatures stay valid.
byte[] signed = PdfSigner.Sign(File.ReadAllBytes("in.pdf"), new PdfSignatureOptions(cert));
```

Signatures are PAdES baseline (`ETSI.CAdES.detached`) by default, or classic `adbe.pkcs7.detached` with `Format = PdfSignatureFormat.Pkcs7Detached`; RSA and ECDSA keys with SHA-256/384/512. The signature is appended as an incremental update, and it works together with `Security` (encrypted files) and `Conformance` (PDF/A: the visible box is drawn as outlines, so it needs no unembedded font).

The CMS structure is built with .NET's own `System.Formats.Asn1` and `System.Security.Cryptography`; no extra package is needed. Signing adds a few milliseconds (the RSA or ECDSA operation and re-reading the file); conversions without a signature are unaffected.

Custom properties are written to the document information dictionary, where Acrobat shows them under Document Properties > Custom and `pdfinfo -custom` lists them. For PDF/A they are also written to the XMP metadata (Acrobat's `pdfx` namespace, with the extension schema PDF/A requires), alongside Keywords, Creator and Producer.

Merging PDFs

`PdfMerger` combines PDFs into one, any PDFs, not only those PdfPecker made:

```
byte[] merged = PdfMerger.Merge(invoicePdf, termsPdf, attachmentPdf);

// With passwords for encrypted inputs (by position), and properties for the result.
byte[] merged2 = PdfMerger.Merge(new[] { a, b }, new PdfMergeOptions
{
    Passwords = new string?[] { null, "secret" },
    Title = "Payslips October",
    CustomProperties = { ["RunId"] = "42" },
});

PdfMerger.MergeFiles("all.pdf", new[] { "a.pdf", "b.pdf" });
```

The pages keep their content, fonts, images, links, annotations, layers and fillable form fields, and each input's bookmarks follow each other. A form field named like one in an earlier input is renamed `name_2`, `name_3` and so on, so the copies keep their own values. Document properties come from the first input unless `PdfMergeOptions` sets them. The result is never encrypted and has no signatures (merging would break them); PDF/A conformance, attachments, page labels and tags are not carried over. Images, fonts and other resources that are byte-for-byte the same in several inputs, such as a logo on every payslip of a run, are written once.

Inputs are handled as untrusted: damaged files are repaired where possible, decoding is limited per input, and `MaxPages` (default 100,000) caps the result. Inputs without any pages throw `InvalidDataException`. Merging was tested on PDFs from Chrome (as IronPDF makes them), Ghostscript, LibreOffice, MuPDF, qpdf, ReportLab, pypdf incremental updates, scanners (JPEG, CCITT G4, CMYK, OCR text layers) and damaged files (wrong or missing cross-reference tables, truncated files, junk before the header); every merged page rendered pixel-identical to its original in Poppler. Merging four small documents takes about 1 ms, and 1,000 one-page Chrome payslips about 1 s.

PDF to image

`PdfToImage` and `PdfRenderer` render PDF pages to PNG (or raw RGBA pixels). This works for any PDF, not only PdfPecker's own output, and also uses no third-party code: the PDF parser, font engines, JPEG and fax decoders, rasterizer and PNG encoder are all part of the library.

```
// Every page as PNG bytes (rendered in parallel), or one page (0-based index)
byte[][] pages = PdfToImage.Convert(pdfBytes, new ImageOptions { Dpi = 150 });
byte[] first = PdfToImage.ConvertPage(pdfBytes, 0);

// report.pdf -> out/report-1.png, out/report-2.png, ...
PdfToImage.ConvertFile("report.pdf", "out");

// Open once, render many pages; RenderedImage also exposes the RGBA pixels
var renderer = PdfRenderer.Open(pdfBytes, password: null);
RenderedImage image = renderer.RenderPage(2, new ImageOptions { Width = 800 });
image.SavePng("page3.png");
```

`ImageOptions` sets `Dpi` (default 150) or a target `Width`/`Height` in pixels, `BackgroundColor` or `Transparent`, `RenderAnnotations`, `Password`, `MaxDegreeOfParallelism` and `PngCompression`. A `PdfRenderer` is thread-safe and shares decoded fonts and images between pages. Encrypted files open with the user or owner password (`PdfPasswordException` otherwise); unreadable files throw `InvalidDataException`, as does a page that needs more than 20 million drawing operations (nested forms can multiply a few bytes into billions). Pass a cancellation token to bound the time a page may take.

What the renderer supports:

- **Files**: cross-reference tables and streams, object streams, incremental updates, damaged files (rebuilt by scanning), RC4 and AES-128/256 encryption, all standard filters (Flate, LZW, ASCII85, ASCIIHex, RunLength, predictors).
- **Graphics**: paths, fills (non-zero and even-odd), strokes with joins, caps and dashes, clipping, anti-aliasing, all blend modes, constant alpha, soft masks, transparency groups, axial, radial, function and mesh shadings (types 1 to 7), tiling patterns, form XObjects, optional content (layers) and annotation appearances.
- **Text**: embedded TrueType, OpenType, CFF and Type 1 fonts, Type 3 fonts, CID fonts with CMaps, vertical text, all text rendering modes. The 14 standard fonts and non-embedded fonts are drawn with built-in Liberation and DejaVu-derived faces stretched to the document's widths.
- **Images**: every bit depth and colour space (Gray, RGB, CMYK, Lab, ICC-based, Indexed, Separation, DeviceN), baseline and progressive JPEG, JPEG 2000 (`JPXDecode`, including embedded alpha), JBIG2 (generic, text, symbol, refinement and halftone regions, with `JBIG2Globals`), CCITT fax, stencil masks, colour-key masks and soft masks.

For what the renderer does not support yet, see Limitations.

Website

`web/PdfPecker.Web` is a small ASP.NET Core site for converting HTML and Markdown in the browser: paste or open an `.html` or `.md` file (or drag one onto the page), pick page size, orientation, margins and page numbers, and preview or download the PDF. Markdown can also be converted to HTML.

```
dotnet run --project web/PdfPecker.Web -c Release        # then open the URL it prints
docker build -f web/PdfPecker.Web/Dockerfile -t pdfpecker-web . && docker run -p 8080:8080 --
memory 2g pdfpecker-web
```

It also exposes the converter as an HTTP API:

```
curl -X POST "http://localhost:8080/api/convert?pageSize=Letter&pageNumbers=true" \
     -H "Content-Type: text/html" --data-binary @invoice.html -o invoice.pdf
```

Or upload a file as a form:

```
curl -X POST http://localhost:8080/api/convert -F "file=@invoice.html" -F pageSize=A4 -F
pageNumbers=true -o invoice.pdf
```

`POST /api/pdf-to-image` turns a PDF into PNGs: one page with `?page=N` (or when the PDF has one page), otherwise a zip with a PNG per page. Options: `dpi` (10 to 300, default 150), `width`, `height` and `password`.

```
curl -X POST "http://localhost:8080/api/pdf-to-image?page=1&dpi=200" --data-binary @report.pdf
-o page1.png
curl -X POST http://localhost:8080/api/pdf-to-image -F "file=@report.pdf" -o pages.zip
```

`POST /api/convert` also accepts JSON: `{ "html": "...", "pageSize": "A4", "landscape": false, "marginMm": 12.7, "pageNumbers": true }`. Add `watermark` (text stamped on every page), `password` (AES-256, needed to open the PDF) or `pdfa: true` (PDF/A-2b) in any of the three forms; send passwords in the form or JSON body rather than the query string, which servers tend to log.

Markdown works the same way. The input is read as Markdown when it is uploaded as a `.md`/`.markdown` file, sent as `"markdown"` in JSON or as a `markdown` form field, posted with `Content-Type: text/markdown`, or marked with `format=markdown`. Add `output=html` to get the HTML page instead of a PDF:

```
curl -X POST http://localhost:8080/api/convert -F "file=@README.md" -o README.pdf
curl -X POST http://localhost:8080/api/convert -F "file=@README.md" -F output=html -o
README.html
curl -X POST http://localhost:8080/api/convert -H "Content-Type: application/json" -d
'{"markdown":"# Hi","output":"html"}'
```

The site never reads files from the server, limits requests to 5 MB, 15 seconds, 1,000 pages and 120 conversions per minute per IP address (per /64 network for IPv6), runs a few conversions at a time (answering 503 when busy or low on memory), and stores nothing. Give the container a memory limit (`--memory 2g`): the number of simultaneous conversions follows it, and a document that would need more is refused instead of affecting the host. Behind a reverse proxy or CDN, set `TrustedProxies` to the proxy's addresses or networks (for example `-e TrustedProxies=10.0.0.0/8`) so the rate limit applies to each visitor rather than to the proxy. The page sends a strict Content-Security-Policy (no inline script) along with the usual security headers.

What is supported

HTML: the common document elements: headings, paragraphs, `div`/`section`-style blocks, `span`, `a` (external links and `#id` links inside the document), `b`/`strong`, `i`/`em`, `u`, `s`, `sub`, `sup`, `small`, `big`,

`mark`, `code`, `pre`, `blockquote`, `hr`, `br`, `ul`/`ol`/`li` (nested, with `start` and `value`), `dl`, `table` (with `thead`, `tbody`, `tfoot`, `caption`, `colspan`, `rowspan`, `col width`), `img`, inline `<svg>`, and legacy attributes such as `align`, `valign`, `bgcolor`, `width`, `height`, `border`, `cellpadding`, `cellspacing` and `<font>`. The parser is forgiving about unclosed and misnested tags, the way browsers are.

Markdown: CommonMark 0.31 (651 of the 652 spec examples pass; the one miss is an exotic HTML5 entity name) plus the GitHub Flavored Markdown extensions: tables with column alignment, `~~strikethrough~~`, task list items (`- [x]`) and bare `www.`/`https://`/e-mail autolinks. Headings get GitHub-style ids, so `[see below](#heading-text)` links work in the PDF too. The parser runs in linear time on hostile input (deep nesting, unclosed brackets, long delimiter runs). See Limitations for what is not supported.

CSS: `<style>` blocks, `<link rel="stylesheet">`, inline `style` attributes, `@media print`, and `@page`. Selectors: type, `.class`, `#id`, `*`, attribute selectors (`[a]`, `=`, `~=`, `|=`, `^=`, `$=`, `*=`), descendant, `>`, `+`, `~`, `:first-child`, `:last-child`, `:only-child`, `:nth-child()`, `:nth-last-child()`, `:not()`, `:root`, `:link`, `:empty`, with specificity and `!important` handled as in browsers. Properties:

Area	Properties
Box model	`display` (block, inline, inline-block, flex, inline-flex, grid, inline-grid, list-item, table parts, none), `margin`, `padding`, `border` (solid, dashed, dotted, double), `border-radius` (incl. elliptical `/` radii and percentages; rounds backgrounds, borders, shadows and images), `box-shadow` (outer shadows; a blur is drawn as fading bands), `overflow` (`hidden`, `clip`, `auto` and `scroll` clip to the padding box, following its rounded corners; `overflow-x`/`overflow-y`), `width`, `height`, `min-`/`max-width`, `min-height`, `box-sizing`
Text	`color`, `font`, `font-family`, `font-size`, `font-weight`, `font-style`, `line-height`, `text-align` (incl. justify), `text-decoration`, `text-transform`, `text-indent`, `letter-spacing`, `word-spacing`, `white-space`, `overflow-wrap`/`word-wrap`, `word-break` (`break-all`), `vertical-align`. Lines break as in Chrome: at spaces, after hyphens and dashes, at `` (showing a hyphen) and ``; a word too wide for its line overflows unless `overflow-wrap` or `word-break` allows breaking it. `line-height: normal` comes from the font's own metrics, rounded to whole pixels as Chrome does, and border widths snap to whole pixels. Text is kerned as in Chrome: from the font's GPOS or `kern` table, and for the standard fonts from the Liberation fonts browsers use in their place. Embedded fonts get their standard ligatures (GSUB `liga`/`clig`, such as fi and ffi) as in Chrome, except in letter-spaced text; `font-variant-ligatures: none` or `font-feature-settings: "liga" 0` turns them off
Backgrounds	`background-color`, `background-image` and `background` with `url()` images (PNG with alpha, JPEG), `linear-gradient()`, `radial-gradient()`, `repeating-linear-gradient()` and `repeating-radial-gradient()` (several layers allowed); `background-repeat` (incl. `space`, `round`), `background-position` (incl. edge offsets like `right 10px bottom 5px`), `background-size` (`cover`, `contain`, lengths), `background-origin`, `background-clip`. Repeated images are written once as PDF tiling patterns. `url()` in a linked stylesheet is relative to the stylesheet and follows the same file access rules as ``. As in Chrome's print output, the background of `<html>`, or else of ` `, fills every page area (images and gradients too, positioned against the root box and continuing from page to page); `@page { background: }` also colours the page margins. `opacity` on SVG
Flexbox	`flex-direction`, `flex-wrap`, `flex-flow`, `flex`, `flex-grow`/`-shrink`/`-basis`, `order`, `justify-content`, `align-items`, `align-self`, `align-content`, `gap`, `margin: auto`

Area	Properties
Grid	`grid-template-columns` / `-rows` (lengths, `%`, `fr`, `auto`, `min-content`, `max-content`, `minmax()`, `fit-content()`, `repeat()` incl. `auto-fill` / `auto-fit`), `grid-auto-rows` / `-columns`, `grid-auto-flow`, `grid-template-areas`, `grid-template`, named lines (`[name]`), `grid-column` / `-row` (line numbers, names and `span`), `grid-area` (incl. area names), `gap`, `justify-items` / `-self`, `align-items` / `-self`, `justify-content`, `align-content`, `place-*`
Columns	`column-count`, `column-width`, `columns`, `column-gap`
Variables	custom properties (`--name: value`) and `var(--name, fallback)`, anywhere including shorthands
Positioning	`float` and `clear`; `position` (`relative`, `absolute`, `fixed`; `sticky` is drawn as `relative`) with `top`, `right`, `bottom` and `left`; `z-index`
Transparency	`opacity` (a block is painted as one PDF transparency group; an inline `<span>` fades its text and background), translucent colours everywhere (`rgba()`, `hsla()`, `#rrggbbaa`, `transparent`), gradients that fade to `transparent` (as a soft mask). PDF/A-2 and 3 allow all of this
Transforms	`transform` (`translate()`, `translateX()` / `Y()`, `scale()`, `scaleX()` / `Y()`, `rotate()`, `skew()`, `skewX()` / `Y()`, `matrix()`) and `transform-origin`
Generated content	`::before` and `::after` with `content` (strings, `attr()`, `counter()`, `counters()`, `open-quote` / `close-quote`), `counter-reset`, `counter-increment`, `counter-set`
SVG	`fill`, `stroke`, `stroke-width`, `fill-opacity`, `stroke-opacity`, `fill-rule`, `stroke-linecap`, `stroke-linejoin`, `stroke-miterlimit`, `clip-path`, `clip-rule`, `text-anchor`, `dominant-baseline` (as CSS or presentation attributes)
Lists	`list-style-type` (disc, circle, square, decimal, decimal-leading-zero, alpha, roman, none), `list-style`
Tables	`border-collapse` (each shared border drawn once, the wider one winning), `border-spacing`, cell `vertical-align`, automatic column widths, `table-layout: fixed`, a table `height` shared out over the rows
Paging	`page-break-before` / `after` / `inside`, `break-before` / `after` / `inside` (`always`, `page`, `avoid`), `orphans`, `widows`, `@page` size and margins

Units: `px`, `pt`, `em`, `rem`, `%`, `mm`, `cm`, `in`, `pc`, `ex`, `ch`, `vw`, `vh`, `vmin`, `vmax` (the viewport is the page's content area, minus any running header and footer, in whole pixels as in Chrome). Colours: names, `#rgb`, `#rrggbb`, `#rrggbbaa`, `rgb()`, `rgba()`, `hsl()`, `hsla()`.

Pagination: lines, images and table rows are never cut in half at a page boundary; table header rows repeat on every page a table spans; backgrounds and borders continue across pages. Flex lines and grid rows move to the next page as a whole when they don't fit. A paragraph that splits leaves at least `orphans` lines (default 2) at the bottom of a page and carries at least `widows` lines (default 2) to the next. Headings keep with the content that follows them (`break-after: avoid` in the default styles), and `break-before` / `break-after: avoid` does the same for any block. `break-inside: avoid` keeps a block or a whole table on one page when it fits.

Links and bookmarks: `<a href>` becomes a clickable link, whether inline or a block (a card wrapped in a link), and stays clickable on both pages when a page break splits it. External URLs open in the viewer's browser and `#id` links jump to the element. Headings `h1`-`h6` become the PDF's bookmarks, nested by level; set `PdfOptions.Bookmarks = false` to leave them out.

Running header and footer: an element with the `data-sticky-pdf-header` or `data-sticky-pdf-footer` attribute is taken out of the flow and drawn at the top or bottom of every page, and the page

content flows in the space between. Inside it, `{page}` and `{total-pages}` (or `{pages}`) are replaced with the page number and page count. An optional `height` attribute (`height="17mm"`) reserves at least that much space.

```
<header data-sticky-pdf-header height="17mm">ACME Ltd · Payslip</header>
<main>...</main>
<footer data-sticky-pdf-footer height="12mm">Page {page} of {total-pages}</footer>
```

SVG: inline `<svg>` with `path` (all commands, arcs included), `rect` (incl. rounded), `circle`, `ellipse`, `line`, `polyline`, `polygon`, `text` and `tspan` (`x`, `y`, `dx`, `dy`, `text-anchor`, `dominant-baseline`, standard fonts), `g`, `use`, nested `svg`, `viewBox`, `preserveAspectRatio` and `transform`, drawn as vector paths. `linearGradient` and `radialGradient` (both units, `gradientTransform`, `spreadMethod`, `href` inheritance) become PDF shadings, and `clipPath` clips the elements that reference it.

Images: JPEG (embedded as-is, RGB, grayscale and CMYK) and PNG (all colour types and bit depths, transparency, interlacing), from files, `data:` URIs or your `ResourceResolver`. Opaque PNGs are passed through without re-encoding. Images wider than the page are scaled down to fit.

Fonts: registered and `@font-face` TrueType/OpenType fonts are embedded as subsets (Type0/CIDFontType2 for TrueType outlines, CIDFontType0 with an OpenType program for CFF outlines) with a ToUnicode map, so text can be searched and copied. Without them, the PDF standard fonts are used: `font-family` maps to Helvetica (sans-serif, Arial, Verdana, ...), Times (serif, Georgia, ...) or Courier (monospace, Consolas, ...), each in regular, bold, italic and bold italic. With `EmbedStandardFonts` or PDF/A, these are replaced by embedded subsets of the bundled, metric-compatible Liberation Sans, Serif and Mono.

Limitations

PdfPecker is built for fast conversion of HTML templates you control, not for rendering arbitrary web pages. These are the known gaps; most are scope choices for speed and size, and they are the first candidates for future work. Anything not listed in What is supported should be assumed missing.

By design

- No JavaScript. Scripts are never run, so content must be in the HTML before conversion (render it server-side first).
- No network access: `http:` / `https:` stylesheets and images are not downloaded. Supply them through your `ResourceResolver`, as files under `BaseDirectory`, or as `data:` URIs.
- Form controls (`<input>`, `<select>`, `<textarea>`, `<button>`), `<canvas>`, `<iframe>`, `<video>`, `<audio>`, `<object>` and `<embed>` are not drawn; the PDF has no fillable form fields.

Text and fonts

- Without registered fonts or `EmbedStandardFonts`, text uses the standard PDF fonts, so characters are limited to the Windows-1252 set (Western European languages); other characters are replaced with `?`. The bundled Liberation fonts add Central European, Greek and Cyrillic, but not CJK or other scripts: register fonts for those.

- No complex-script shaping: Arabic, Hebrew, Indic and other scripts that need contextual forms or reordering are not laid out correctly, and there is no right-to-left or vertical text (`direction`, `writing-mode`). Shaping is limited to pair kerning and standard ligatures; no contextual alternates, small caps or other OpenType features.
- Variable fonts use their default instance (`font-variation-settings` and `font-stretch` are ignored).
- No automatic hyphenation (`hyphens: auto`); words break only at spaces, hyphens, `` and ``.
- SVG `<text>` uses the standard fonts (embedded as Liberation for PDF/A), only the first value of position lists and no `textPath`.

CSS

- No `calc()`, `min()`, `max()` or `clamp()`; a declaration using them is ignored.
- `@import` is ignored (use `<link rel="stylesheet">`). Rules inside `@supports`, `@layer` and `@container` blocks are skipped, as are `@page` `:first` / `:left` / `:right` and page margin boxes (`@top-center`; use running headers and footers instead).
- Selectors: no `:first-of-type`, `:last-of-type`, `:nth-of-type()`, `:is()`, `:where()`, `:has()` or `::first-letter` / `::first-line` / `::marker`. A rule with an unsupported selector is skipped. `:hover`, `:focus` and similar interaction states never match.
- Properties not supported: `text-shadow`, `outline`, `text-overflow` (no ellipsis), `filter`, `mix-blend-mode`, `clip-path` on HTML elements (only in SVG), `border-image`, `object-fit`, `aspect-ratio`, `list-style-image`, `column-span`, `column-rule` and inset `box-shadow`.
- Borders: `groove`, `ridge`, `inset` and `outset` draw as solid, and so do `double` sides on rounded corners.
- `position: sticky` is drawn as `relative`; `background-attachment: fixed` scrolls with the box.
- Lists: `list-style-type` covers disc, circle, square, decimal, decimal-leading-zero, alpha and roman only (no Greek, CJK or `@counter-style`).

Backgrounds and transparency

- No `background-clip: text`; SVG files as `url()` backgrounds are not drawn; backgrounds of inline elements (`<span>`) show only their colour, not images or gradients.
- Blurred box shadows and repeating gradients with translucent stops are blended onto the background colour rather than truly see-through; SVG gradient stops with transparency are blended onto white.

Layout

- Grid: no `dense` packing and no subgrid; `span <name>` counts as `span 1`.
- Multi-column: children are kept whole and balanced between columns; a single block is never split across columns.
- Overflow: table cells do not clip their content; a positioned child with a `z-index` inside a clipping box stays stacked within that box.

Images and SVG

- Images: JPEG and PNG only; GIF, WebP, AVIF, BMP and SVG files in `` are not drawn.
- Inline SVG: no embedded `<image>`, masks, patterns, markers or filters.

Markdown and the PDF renderer

- Markdown: no footnotes, definition lists, math or front matter.
- `PdfRenderer` (PDF to image): no predefined CJK CMaps or non-embedded CJK fonts, and ICC profiles are not applied (ICC-based colours use their device equivalents).

Performance

Measured with `bench/PdfPecker.Benchmarks` on a 4-core cloud VM (.NET 10, Release, after JIT warm-up):

Document	Pages	Median	Throughput
Invoice with a PNG logo and a styled table	1	0.55 ms	about 1,800 docs/s on one core
Feature tour (text, lists, tables, JPEG and PNG)	5	2.5 ms	about 1,700 docs/s on 4 cores
Dense report with 1,000 table rows	30	19.5 ms	about 1,500 pages/s on one core
Markdown guide (`guide.md`, headings, lists, code, a table) to PDF	2	0.9 ms	about 1,000 docs/s on one core

Parsing Markdown to HTML runs at roughly 20 to 45 MB/s, so it is a small part of a Markdown to PDF conversion.

The first conversion in a process also pays for JIT compilation (about 150 ms); later ones run at the speeds above.

Rendering PDF pages to images, same machine, one core per page (`PdfRenderer`, first render of each page of a freshly opened document, so fonts and images are decoded too):

Document	Render at 96 DPI	Render at 150 DPI	PNG encoding at 150 DPI
Invoice (1 page)	13 ms	19 ms	24 ms
Feature tour (5 pages)	7 ms per page	12 ms per page	26 ms per page
Large report (30 pages)	5 ms per page	10 ms per page	29 ms per page

Rendering the same page again (fonts and glyphs cached) takes about 5 ms at 150 DPI. Multi-page conversions render pages in parallel.

Building

```
dotnet build
dotnet test
dotnet run --project samples/PdfPecker.Sample -c Release      # converts
samples/PdfPecker.Sample/html/*.html and *.md into ./output
dotnet run --project bench/PdfPecker.Benchmarks -c Release
dotnet run --project bench/PdfPecker.Benchmarks -c Release -- --requirements    # memory and
CPU per conversion
dotnet pack src/PdfPecker -c Release -o ./nupkg
```

How it works

Markdown is first turned into HTML: `Markdown/BlockParser` finds the block structure line by line, following the CommonMark reference algorithm, `Markdown/InlineParser` handles emphasis (delimiter stack) and links (bracket stack), and `Markdown/HtmlRenderer` writes the HTML, which then takes the same path as any other document:

1. `Html/HtmlParser` builds a DOM in a single pass, applying the browser rules for implied end tags.
2. `Css/StyleResolver` cascades a user-agent stylesheet, presentational attributes, document stylesheets and inline styles. Rules are indexed by id, class and tag, and elements with identical inputs share one computed style.
3. `Layout/LayoutEngine` lays out blocks, inline text (line breaking, justification), lists and tables (automatic column widths), paginating as it goes, and produces a flat list of draw operations.
4. `Pdf/PdfDocumentWriter` writes PDF 1.4: content streams, fonts, image XObjects, link annotations, the bookmark outline and the cross-reference table. `Fonts/FontSubsetter` and `Fonts/CffSubsetter` cut embedded fonts down to the glyphs used (with their composite components and the CFF subroutines they call).

The renderer (`Rendering/`) works the other way round: `Parsing/PdfFile` reads the object graph, `PageRenderer` interprets content streams, `Fonts/` turns glyphs into outlines (cached as anti-aliased bitmaps), `Images/` decodes images, and `Graphics/Rasterizer` fills paths with exact area coverage before `PngEncoder` writes the result.

Third-party font data built into the renderer is listed in THIRD-PARTY-NOTICES.md.